

RAGDoll: Efficient Offloading-based Online RAG System on a Single GPU

Weiping Yu*
Ningyi Liao*
weiping001@e.ntu.edu.sg
liao0090@e.ntu.edu.sg
Nanyang Technological University
Singapore

Siqiang Luo†
Nanyang Technological University
Singapore
siqiang.luo@ntu.edu.sg

Junfeng Liu
Nanyang Technological University
Singapore
junfeng001@e.ntu.edu.sg

Abstract

Retrieval-Augmented Generation (RAG) enhances large language model (LLM) generation quality by incorporating relevant external knowledge. However, deploying RAG on consumer-grade platforms is challenging due to limited memory and the increasing scale of both models and knowledge bases. In this work, we introduce RAGDOLL, a resource-efficient, self-adaptive RAG serving system integrated with LLMs, specifically designed for resource-constrained platforms. RAGDOLL exploits the insight that RAG retrieval and LLM generation impose different computational and memory demands, which in a traditional serial workflow result in substantial idle times and poor resource utilization. Based on this insight, RAGDOLL decouples retrieval and generation into parallel pipelines, incorporating joint memory placement and dynamic batch scheduling strategies to optimize resource usage across diverse hardware devices and workloads. Extensive experiments demonstrate that RAGDOLL adapts effectively to various hardware configurations and LLM scales, achieving up to 3.6 $\times$ speedup in average latency compared to serial RAG systems based on vLLM.

ACM Reference Format:

Weiping Yu, Ningyi Liao, Siqiang Luo, and Junfeng Liu. 2018. RAGDoll: Efficient Offloading-based Online RAG System on a Single GPU. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. <https://doi.org/XXXXXX.XXXXXX>

1 Introduction

The emergence of large language models (LLMs) [1, 8, 58] has transformed diverse aspects of people's daily lives and production, fostering applications including programming assistants [10, 20] and universal chatbots [22, 45]. To ensure the quality and relevance of LLM responses in context-rich scenarios, Retrieval Augmented Generation (RAG) [5] has become popular for invoking external knowledge bases, such as local and private data in companies [7, 9, 40, 44], into LLM inference. Local RAG utilizations, typically serving a small number of concurrent queries, focus on responding with low overall latency across these batches of data [17, 50, 61].

*Both authors contributed equally to this research.

†Corresponding Author

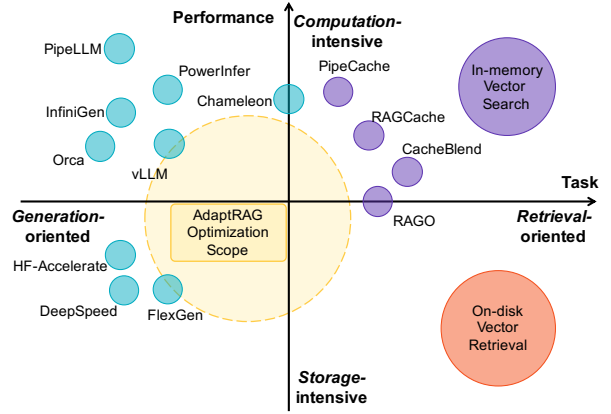


Figure 1. Representative techniques related to RAG serving, depicted by corresponding tasks and design objectives.

In general, a RAG workflow first *retrieves* data from the knowledge database according to user requests, then *generates* responses by the LLM based on the extracted information. The integration of retrieval-based techniques, however, poses new challenges for comprehensive performance enhancements, especially as RAG systems are commonly deployed in environments with limited computational resources. This issue stems from the inherent tension between retrieval and generation components, which exhibit different characteristics in terms of computation needs and memory usage patterns. For example, while the retrieval operation benefits from higher database throughput through batch processing with larger batch sizes [11, 43], the generation operation usually demands a limited batch size to ensure low latency considering LLM tensor placement [33, 53, 55].

In Figure 1, we present existing studies representing LLM serving, RAG computation, and database retrieval optimization techniques from two dimensions: their intended *tasks* focusing on retrieval or generation processes, and the *performance bottleneck* in relation to computation and storage resources. Since most of these techniques focus only on a single component or design objective within the RAG system, directly applying them for RAG deployment on resource-constrained platforms leads to issues from two perspectives.

First, the RAG workflow features various data placements *across devices*, especially concerning the GPU-CPU-disk storage hierarchy [53]. LLM serving frameworks usually leverage offloading techniques [4, 23, 33, 55] in consumer-grade environments, transferring idle data to RAM and disk to save GPU memory. Vector databases used for knowledge retrieval also actively maintain partial or full-scale data in memory [6, 14, 41] or on disk [11, 24, 59]. While both components tend to allocate more main memory for better performance, the available memory capacity is relatively limited compared to the sizes of the LLM and database. Excessive overhead risks memory thrashing or even exhaustion, resulting in impeded response latency.

Second, the processing and transmission of data *over time* need to be carefully designed. The RAG pipeline manipulates a batch of data to be first computed from knowledge embeddings [19, 35] by the CPU during retrieval, and then transferred to the GPU for generation. Additionally, the offloading strategy periodically relocates LLM tensors between the GPU and CPU. These operations exhibit intricate dependencies. Current studies on RAG serving enhancements [30, 64] predominantly employ a straightforward serial processing strategy, which relies on the smooth execution of every phase. However, our evaluation reveals that overall latency is undermined by insufficient data utilization when computation and memory resources are constrained. Hence, a comprehensive pipeline is necessary to minimize redundant data processing, waiting, and transmission time, while ensuring concurrent utilization of various devices.

In this paper, we propose RAGDOLL, a system for RAG deployment that integrates LLM and database components on a single consumer-grade GPU. As illustrated in Figure 1, RAGDOLL encompasses a wide design space for collaboratively enhancing both computation and storage utilization, exploiting the benefits of different serving models adaptively toward lower latency. Specifically, RAGDOLL highlights three key designs for addressing the above challenges.

To effectively utilize storage across devices, RAGDOLL introduces an innovative *joint memory placement* scheme integrating both retrieval and generation components. The system strategically optimizes placement across the entire storage hierarchy involving GPU, CPU, and disk, accounting for their distinct capacities and access speeds. This comprehensive maneuver improves device usage, achieving 2× improvement in average CPU utilization in practice compared to conventional serial RAG implementations. The dominant memory overhead elements, including database partitions and LLM tensors, can be coordinated through adjusting pivotal configuration parameters in RAGDOLL.

To enhance data utilization, RAGDOLL proposes a novel *multi-pipeline RAG integration* that decouples CPU-bound retrieval and GPU-bound generation stages. This parallel execution on different computational devices effectively reduces the idle time from 80% to 30% of the overall latency in

the RAG workflow. Furthermore, the fine-grained data flow manipulation loosens the component interdependencies, enabling a broader range of pipeline configurations.

Lastly, the joint and pipelined designs bring about the *adaptive scheduling* ability of RAGDOLL for serving scenarios where static schemes fall short. Thanks to the dedicated data placement and pipeline execution modules, the impact on system workload can be effectively adjusted through a few performance-related parameters, especially batch size and resource allocation. The configurations in RAGDOLL serving are designed to be dynamically accommodated to the status of incoming queries and device utilization. To this end, RAGDOLL can adapt to different serving requirements by selecting the most suitable scheme dynamically, achieving peak latency reductions of up to 80%.

We implement a RAGDOLL prototype, integrating the advanced FlexGen [53] as the offloading-based LLM serving system and Milvus [43] as the high-performance vector database. We extensively evaluate its performance against popular RAG systems [19, 35] deployed with LLM frameworks [23, 33] for consumer-grade platforms. Specifically, RAGDOLL is able to serve 8B-70B models under dynamic workload conditions, using only a single GPU with 12-24GB of memory and 176-256GB of main memory. Experimental results demonstrate that RAGDOLL reduces waiting and generation times by up to 20× and 5×, respectively, achieving up to 3.6× and 11.7× boosts compared to baselines in overall average latency.

Our contributions can be summarized as follows:

- We propose RAGDOLL, a novel RAG system tailored for on-demand serving under memory constraints with a comprehensive design comprising computation and storage management for database retrieval and LLM generation.
- We introduce multi-pipelining for on-demand RAG serving. The novel LLM prefetching enables continuous enqueueing and dynamic data placement.
- We utilize adaptive configuration with online optimization, which comprehensively balances retrieval and generation overhead, as well as batch-wise latency.
- We implement RAGDOLL, which outperforms leading RAG deployments in resource-constrained settings by up to 3.6× in average latency.

2 Background

2.1 Offloading-based LLM Inference

Typical LLM inference is performed in an auto-regressive manner [1, 58]. Given a user input, the model generates one token at a time by sequentially processing the data through a series of transformer blocks, namely *layers*. Each layer comprises a self-attention mechanism and a feed-forward network with learnable parameters, namely *weights*, to capture dependencies among tokens. The layer also manipulates a *KV cache*, which stores previously computed key-value

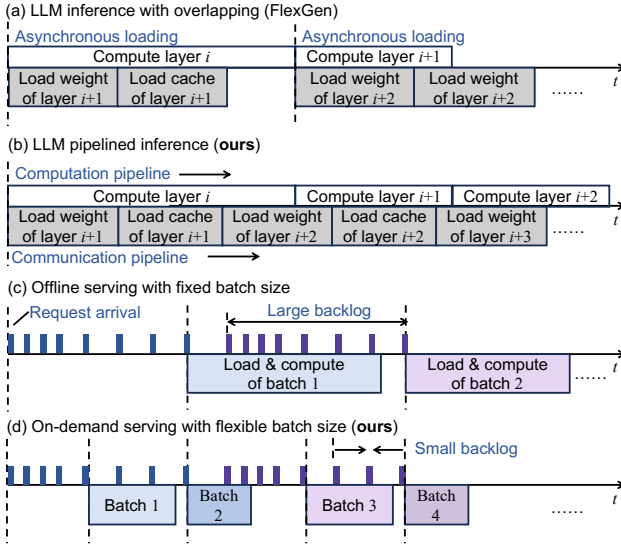


Figure 2. Pipelines in memory-intensive RAG systems: (a) Standard overlapping LLM inference may misalign computation and prefetching due to CPU scheduling and compute jitter. (b) Our LLM pipeline separates computation and communication for continuous prefetching. (c) Fixed batch scheduling accumulates larger backlogs under memory-intensive conditions. (d) Our backlog-aware batch scheduling adjusts flexibly to minimize backlogs.

results in the attention module to reduce redundant computation throughout generation. As the data propagate through these layers, the model accumulates contextual information necessary for accurate token prediction.

LLMs have seen a rapid growth in size in recent advancements, often exceeding the memory capacity of a single GPU [55]. To alleviate the device requirement and enable LLM inference in consumer-grade platforms, offloading techniques [4, 23, 33, 53, 55] excel at transferring portions of the LLM tensors, including weights and KV caches, to RAM and disk, thereby bounding the GPU memory footprint. Figure 2(a) depicts the predominant practical LLM serving timeline [51, 53]. Upon computing the layer i , the system *prefetches* the tensor of the next layer $i+1$ asynchronously. Communication time between GPU and host can be partially overlapped with computation to reduce inference latency.

However, in RAG applications, the memory occupation and transfer bandwidth are also vital for the retrieval performance. Irregular placement of model weights and cache may result in memory thrashing or congestion, which compels the system to resort to the slower disk storage and hinders the overall performance.

2.2 On-demand RAG Serving

RAG presents a promising solution for supplying vast information from external knowledge bases to LLM inference, effectively expanding its contextual understanding and generation quality without retraining LLM. The retrieval ability

is built on vector databases, where knowledge such as documents is processed into data chunks and embedded into vectors. During retrieval, these vectors are matched with the embedding of the given query through similarity search, and corresponding chunks with high relevance are used for subsequent LLM generation.

Contemporary knowledge bases are advantageous for managing massive volumes of data [31, 32] with embedding vectors spanning hundreds of gigabytes. This necessitates the on-disk storage scheme [11, 59], where the database stores the majority of data chunks on disk, and only *caches* relevant partitions to main memory for retrieval on demand. Although such a hierarchical storage layout allows for handling a larger scale of information within limited memory, the data transmission becomes another latency bottleneck, which is decisive for all succeeding RAG computations.

Canonical RAG frameworks [34, 37] often encompass the *offline* serving scenario, which handles incoming retrieval requests successively to guarantee individual end-to-end latency. However, due to vast knowledge base size and autoregressive nature of LLMs, processing times of both retrieval and generation are not bounded. In *online* circumstances [30, 33, 64], where concurrent queries occur, requests must wait for the preceding ones to release resources. Consequently, the overall latency among all queries is not satisfactory.

Recent works improving RAG serving [27, 29, 30, 64, 69] mostly focus on the computation-intensive components in the system, assuming abundant memory resources and efficient online serving. The above temporal and device-wise challenges are highly underexplored in these works. We therefore regard them as orthogonal to this study.

2.3 Batch Scheduling

Batching strategies play a crucial role in optimizing the performance tradeoff between throughput and latency in both LLM generation and vector retrieval in RAG systems. In LLM serving architectures, these strategies typically fall into two categories: online and offline processing. Online systems like vLLM [33] implement iteration-level scheduling by allocating a predefined maximum batch size and dynamically adjusting batch composition to minimize average latency. Conversely, offline systems like FlexGen [53] prioritize throughput by employing larger batch sizes.

Modern vector databases [11, 14, 43] support efficient batch processing of multiple embedding queries, allowing them to handle high query volumes while maintaining low latency. For example, Milvus allows sending a batch of vectors in one API call instead of processing them individually. This minimizes network round-trips and leverages bulk processing optimizations in the database engine.

Existing RAG serving frameworks [30, 52, 64] leverage LLM batching strategies to optimize overall performance. Systems like RAGCache [30] and CacheBlend [64] build upon vLLM’s [33] batch scheduling mechanisms to improve

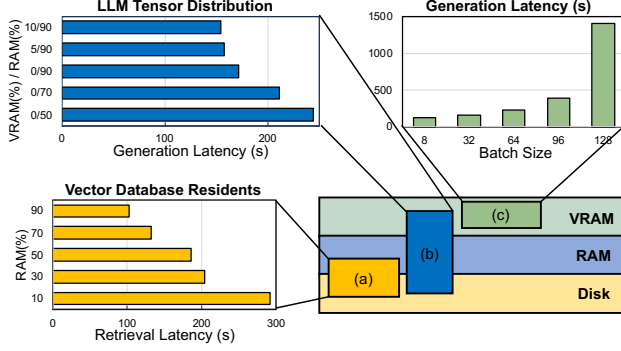


Figure 3. Dissecting an online offloading-based RAG system. (a) LLM tensor placement. (b) vector database residents. (c) compute workspace scheduling.

latency. RAGCache implements a maximum batch size approach to pursue lower average latency, while CacheBlend evaluates performance across different batch scheduling without specifying batch sizes for on-demand scenarios. These approaches demonstrate that effective batching techniques are crucial for achieving low latency in RAG systems.

3 Motivation

3.1 Problem Formulation

In this study, we aim for the problem of resource-constrained RAG serving, which comprises three principal components: a vector database with capacity V , an LLM of size M , and a running cache of size C . Function $\mathcal{P}_\theta(V, M, C)$ returns a d -dimensional vector, representing the allocated memory for the system *components* to d storage devices, which is controlled by the memory placement strategy θ . The LLM execution requires additional storage for hosting the *data*, such as KV cache, which is characterized by function $\mathcal{W}(B, M)$, where B is the generation batch size. The device storage capacity is bounded by vector $D \in \mathbb{R}^d$.

The online serving scenario features incoming query rate as a variable process $\lambda(t)$ through time t . Our objective is to minimize the expected latency $\mathcal{L}(B, \theta, \lambda(t))$ through joint optimization of batch size B and placement strategy θ under the condition of memory bound of each device. This yields the following constrained optimization problem:

$$\begin{aligned} \min_{B, \theta} \quad & \mathcal{L}(B, \theta, \lambda(t)), \\ \text{s.t.} \quad & \mathcal{P}_\theta(V, M, C) + \mathcal{W}(B, M) \leq D, \end{aligned} \quad (1)$$

where the partial ordering $\leq$ indicates component-wise inequality, indicating that the memory overhead on each device, e.g., VRAM, RAM, and disk, remains within its respective capacity constraint.

3.2 Challenges in RAG Systems

Prevailing RAG serving frameworks [27, 30, 64] primarily enhance computation and memory placement on GPU while assuming abundant RAM and small-scale database retrieval. Regarding the LLM component, offloading-based strategies

are directly employed to tackle the device constraints and boost computation. By deploying representative RAG systems on consumer-grade platforms, we observe that the serving performance exhibits a more intricate landscape, rendering new challenges for optimizing latency.

C1: Significant impact from memory-intensive operations. While vLLM [33] facilitates KV caches placement across devices, systems like RAGCache [30] and CacheBlend [64] still require substantial computational resources—two H800 GPUs (2×80GB VRAM) and two A40 GPUs (2×48GB VRAM) respectively—to employ the LLaMA2-70B model as their LLM component. The memory requirements intensify further when handling large-scale knowledge bases that necessitate on-disk vector databases. Consequently, adapting these existing paradigms to common single-GPU environments remains non-trivial. We systematically evaluate this deployment challenge by varying three pivotal factors affecting serving latency: the number of cached partitions in the vector database, the proportion of LLM tensors placed on the GPU, and the generation batch size.

Figure 3 reveals that the respective and overall latency are dominated by configurations on memory placement. Caching more database partitions in RAM from disk can favorably reduce retrieval latency, while an unaggressive offloading strategy for keeping more model tensors on GPU benefits generation efficiency. Notably, the large generation batch size $B = 128$ significantly escalate LLM serving time. This is caused by the increased footprint of KV cache, forcing LLM tensors and vector database partitions to be offloaded to lower memory hierarchies. This observation indicates potential resource race between RAG components when they are not properly configured in a *comprehensive* manner.

C2: Extra waiting time beyond LLM generation. Despite the memory-side overhead, RAG processing essentially demand a considerable running time. Figure 4(a) assesses the device utilization for retrieval on CPU and generation on GPU. The average end-to-end latency within the first batch is approximately 400 seconds, while queries in the second batch reach up to 1000 seconds, which is largely unsatisfactory in practical applications.

We mainly identify two patterns of idle times that increase the latency. The *external* latency is relevant to the backlog phenomenon of batch processing when resources are constrained, in that some requests may already arrive during processing the first batch but have to wait for the previous completion and only be processed in the next batch. This issue affects the end-to-end latency of certain queries and causes long waiting times in extreme cases.

Meanwhile, the *internal* latency roots in the underutilization of GPU and CPU within each batch. Retrieval operations primarily utilize CPU for vector matching in the database, while generation predominantly leverages GPU compute capacity. Hence, without *pipelining*, these devices are left with

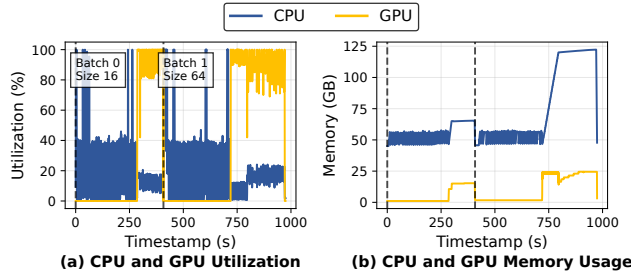


Figure 4. CPU and GPU utilization and memory usage vary in a serial retrieval and generation mode when using different batch sizes under a static memory allocation policy.

long idle times waiting for the computation on the other one, undermining system throughput and efficiency.

C3: Irregular workload of on-demand serving. Unlike offline RAG serving, where system configurations such as batching policies can be predetermined, on-demand use cases must accommodate variable request arrival rates [47, 56, 60]. Limited computational and memory resources further complicate on-demand scheduling. Specifically, fluctuating request patterns during peak and off-peak periods form batches of varying sizes, which entail distinct computational and memory utilization patterns [46, 48]. As illustrated in Figure 4 (b), variations in batch size significantly affect both GPU and CPU memory overhead. For instance, CPU memory utilization increases from approximately 60GB to 120GB during the generation phase. Moreover, retrieval and generation exhibit different characteristics as batch size increases: retrieval time remains relatively constant even at batch size 64, while generation time doubles.

This insight suggests the need for an *adaptive* serving mode that configures memory allocation and batch scheduling based on online workload conditions. However, conventional online batch schedulers such as vLLM do not adequately address the requirements of memory-intensive RAG systems. This mismatch occurs because: (1) retrieval and generation demonstrate different scaling behaviors as batch size increases, making uniform batching across both processes suboptimal; (2) in memory-intensive scenarios where numerous requests are already backlogged before each batch, fine-grained request replacement policies may paradoxically degrade performance [33]. Consequently, we need to carefully model the costs of both retrieval and generation processes, and design backlog-aware batch scheduling strategies.

4 RAGDOLL

In this section, we present RAGDOLL, an adaptive multi-pipeline RAG system for large-scale data and constrained GPU and memory devices. We first outline the new design space offered by RAGDOLL for organizing the RAG workflow comprehensively (Section 4.1). Then, we respectively introduce the novel components and their integration, including the memory placement over devices (Section 4.2) and the

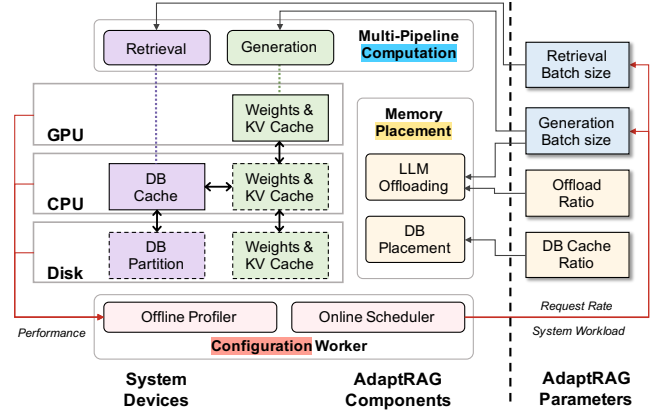


Figure 5. Overview of RAGDOLL.

offloading-based inference systems (Section 4.3). Lastly, we showcase the adaptive tuning of the configuration space through a two-step strategy (Section 4.4).

4.1 Design Space Overview

With the objective to deploy RAG systems in consumer-grade environments with low latency, we attempt to offer a new schema for jointly designing the RAG components, their collaboration, and dynamic configuration during serving. This novel design space not only enhances RAGDOLL through more efficient use of resources, but also contributes to the broader community by exploring how such designs can improve both the effectiveness of retrieval and generation, as well as the balance between computational and storage demands in RAG systems.

RAGDOLL’s novel designs aim to address the critical challenges in Section 3.2. As illustrated in Figure 5, modules in RAGDOLL are tunable during processing, allowing for accommodation to various serving conditions and resources through adjusting pivotal parameters. To showcase the overall design space, we respectively introduce the three key components and their configurable factors in RAGDOLL as follows, while detailed designs are elaborated in later sub-sections.

Joint pipeline: Hierarchical memory placement. RAGDOLL features a novel pipelined scheme, which operates retrieval and generation processes as separate workers. Incoming user requests are first placed into the retrieval queue, which is actively formed as batches for database query. The resulting contexts containing user requests and retrieved document chunks are transferred to the generation queue, where LLM batch processing is performed to generate response texts. Due to the irregular completion time of each batch, the retrieval and generation batches can be formed with different requests, which loosen the dependency of processing orders among them.

The pipelining approach is advantageous for explicit management across different hardware resources. As for *computation*, retrieval operations primarily utilize CPU resources,

while generation tasks leverage GPU capabilities. Decoupling these two pipelines allows for parallel execution with higher device utilization. Regarding *storage*, partitions of the vector database, LLM parameters, and KV caches can be dynamically loaded or offloaded across GPU, CPU, and disk memory hierarchies to ensure efficient computation under capacity constraints.

Offloading prefetch: Dynamic batch size. To specifically deploy RAG with a single GPU, LLM offloading is necessary for dynamically scheduling model weights and KV caches in runtime. RAGDOLL implements an innovative prefetching mechanism in offloading-based LLM serving, which is tailored for the pipeline RAG system. To enhance generation efficiency, we simultaneously conduct computation and communication operations to reduce idle time.

Processing batch size is a critical parameter to balance generation throughput and latency. Our configuration intends to tune the ideal batch size for achieving low latency under varying request arrival rates. Our system further adjusts batch scheduling based on workload patterns in real time to efficiently address processing backlogs. Data placements within the memory hierarchy are adjusted accordingly to accommodate the batch scheduling between generation phases.

Adaptive configuration: Workload feedback. By integrating the tunable components, a configuration layer is applied to adapt the system to compute workload throughout serving. The configuration in RAGDOLL is specifically tuned by two aspects of factors. An active profiler first iteratively explores the *configuration space* to identify optimal settings that minimize average latency while balancing the dual pipelines a priori. During the operational phase, a batch scheduler is designed for selecting the optimal batch size based on the actual *backlog*. Based on the configuration, the system schedules the other components accordingly to reduce average latency across diverse workloads.

4.2 Hierarchical Memory Placement

Conventional RAG workflows exhibit strict dependencies between retrieval and generation phases, creating inefficiencies in data processing and storage. Though pipelining retrieval and generation is not new, we note that implementing effective pipelining under memory-intensive scenarios still presents challenges. RAGDOLL addresses these challenges by comprehensively managing memory allocation across the complete memory hierarchy to prevent resource imbalances and hardware underutilization. This approach optimizes several critical components as shown in Figure 6:

Database partitions. Our system implements sophisticated partition management for vector retrieval, allowing partitions to be dynamically loaded or released according to current operational demands. When the system determines that adjusting the number of resident partitions would optimize performance, it strategically identifies candidates for release

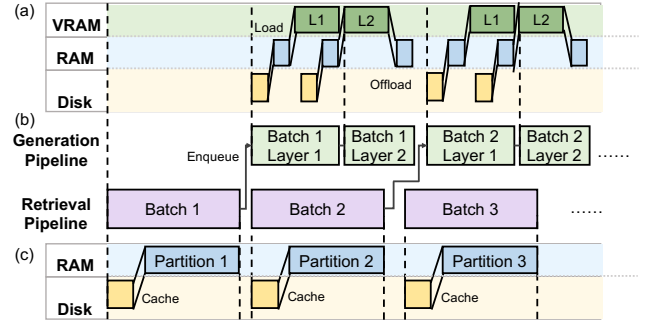


Figure 6. Timelines of computation and memory operations in RAGDOLL: (a) memory placement for LLM weights and KV cache; (b) computation workspace of multi-pipeline; (c) memory placement for database partitions.

and acquisition, executing these operations between consecutive batches of retrieval tasks.

LLM weights and KV cache. For both model weights and KV cache management, we employ a percentage based memory placement strategy that dynamically allocates resources across the memory hierarchy. For instance, if analysis determines an optimal model weight distribution of 50% GPU, 50% CPU, and 0% disk utilization, the system systematically relocates tensors according to this new ratio for the subsequent generation batch. This distribution is governed by configuration that specify the placement of each tensor.

Computation workspace. We implement adaptive computation workspace allocation based on batch size requirements. As batch composition changes, the memory necessary for intermediate computations during model inference must be correspondingly adjusted, including attention matrices and hidden states. The system calculates memory requirements based on the current batch configuration and allocates resources accordingly before processing each generation batch.

Between operational batches, our system employs a sophisticated lock mechanism to ensure that memory reorganization proceeds safely and efficiently. The retrieval and generation workers operate with independent locking protocols, enabling concurrent operation while preventing resource conflicts during configuration updates. Further details on tensor and partition transfer mechanisms can be found in Section 5. This comprehensive approach allows the system to respond dynamically to fluctuating workloads without manual intervention, maximizing resource utilization while maintaining optimal performance characteristics.

4.3 LLM Prefetching Pipeline

RAGDOLL's prefetching module optimizes data movement between GPU, CPU, and disk during offloading-based LLM inference. While existing approaches employ fixed prefetching strategies (Figure 2(a)), they suffer from inefficiencies in resource-constrained environments due to CPU scheduling

and compute jitter [2, 3]. These variations create misalignment between computation and prefetching operations, resulting in resource underutilization. Our enhanced approach (Figure 2(b)) builds upon and enhances this strategy.

Data flow over time. RAGDOLL implements an enhanced asynchronous tensor management system using a thread-based execution model. Rather than restricting prefetching to only the immediate next layer, we continuously enqueue subsequent layers into a prefetching queue—subject only to available memory resources (as illustrated in Figure 2(b)). Dedicated thread pools and multiple CUDA streams manage these concurrent data transfers, ensuring that the required tensors are available for future computation iterations.

This approach effectively minimizes idle time by accounting for computation and loading variations while also dynamically adjusting prefetching strategies based on available memory resources in both prefill and decoding phases. This adjustment is necessary because prefill and decoding phases have fundamentally different memory usage patterns, as demonstrated by FlexGen [53]. During prefill, activation memory requirements are significantly higher since all input tokens must be processed simultaneously, leaving less room for prefetching queues. Conversely, during decoding, only a single new token is processed at each step, resulting in lower activation memory usage and allowing for more aggressive prefetching strategies. Our system leverages this insight to optimize queue capacity, using a more conservative prefetching distance during prefill and increasing prefetching aggressiveness during decoding.

Tensor placement across devices. The prefetching module also integrates closely with the hierarchical memory management described in Section 4.2. It can load and offload tensors across the memory hierarchy and dynamically adjust batch sizes and memory allocations in response to changing workloads. This coordination between components enables RAGDOLL maintaining consistent performance even under challenging operational conditions.

4.4 Adaptive Serving Configuration

To tune the design space, we propose a two-step paradigm. First offline, we employ targeted active profiling to systematically collect performance characteristics of hardware under complex parallel processing; Second online, we provide a backlog-aware batch scheduling approach which is designed to optimize the batch scheduling during the runtime phase.

Active Profiling (Offline). Typical RAG systems often adopt a static configuration, which fails to adapt to different workload characteristics and hardware constraints. To address this limitation, we propose an active profiling approach for preparation that dynamically configures the memory hierarchy. Our method iteratively explores the configuration space to find an optimal balance between the retrieval and generation pipelines. The core insight of our active profiling

approach is that the optimal configuration should balance the latencies of both pipelines. For example, if retrieval time exceeds generation time, we adjust by increasing residential partitions on CPU and reducing LLM tensor allocations on CPU. This balancing act operates under hardware memory constraints that can be expressed as:

$$w_{gpu} \cdot W_{total} + c_{gpu} \cdot C(B) + H(B) \leq M_{gpu} \quad (2)$$

$$w_{cpu} \cdot W_{total} + c_{cpu} \cdot C(B) + P \cdot M_p \leq M_{cpu} \quad (3)$$

where w_{gpu} and w_{cpu} represent the percentage of model weights stored in GPU and CPU memory respectively, W_{total} is the total size of model weights, c_{gpu} and c_{cpu} represent the percentage of KV cache stored in GPU and CPU memory, $C(B)$ is the cache size for batch size B , $H(B)$ is the size of computation workspace, P is the number of resident partitions, M_p is the size of each partition, and M_{gpu} and M_{cpu} represent the available GPU and CPU memory.

Instead of constructing a detailed analytical model, we sample configurations at strategic intervals. While ideally, we would explore all pairs of concurrent retrieval and generation batch sizes to minimize $\max(t_{retrieval}, t_{generation})$, this is computationally expensive. Recognizing that retrieval and generation exhibit different batching behaviors, particularly under memory pressure, we simplify the search. We observe that retrieval cost is dominated by partition loading (see Section 3.2). Therefore, for a fixed number of resident partitions within a given configuration, its value remains nearly constant across varying retrieval batch sizes. Leveraging this, we effectively treat $t_{retrieval}$ as a constant factor and focus the active profiling on tuning the generation batch size. This targeted exploration efficiently identifies the generation batch size that best balances the pipeline latencies, accounting for the complex memory hierarchy interactions more effectively than exhaustive search or purely analytical models.

Backlog-aware Batch Scheduling (Online). Resource-constrained RAG systems are particularly susceptible to overloading, which frequently results in request backlogs that existing scheduling approaches struggle to handle effectively. To specifically address these challenges, we analyze the correlation between batch size and average latency. Let $T(B)$ represent the processing time for a batch of size B :

$$T(B) = a \cdot B^c \quad (4)$$

where $a > 0$ and $c \geq 0$ are constants. For a single batch containing all n requests, the average latency is:

$$L_1 = T(n) - \frac{1}{n} \sum_{i=1}^n t_i \quad (5)$$

When dividing requests into k equal batches of size $\frac{n}{k}$, the average latency becomes:

$$L_k = \frac{k+1}{2} \cdot T\left(\frac{n}{k}\right) - \frac{1}{n} \sum_{i=1}^n t_i \quad (6)$$

The maximum batch size is optimal when $L_1 \leq L_k$, giving:

$$T(n) \leq \frac{k+1}{2} \cdot T\left(\frac{n}{k}\right) \Rightarrow 2 \cdot k^c \leq k+1 \quad (7)$$

For example, when $k = 2$ (splitting into two equal batches):

$$2 \cdot 2^c \leq 3 \Rightarrow c \leq \log_2\left(\frac{3}{2}\right) \approx 0.585 \quad (8)$$

Therefore, using the maximum batch size minimizes average latency when the processing time scales sublinearly with batch size at a rate less than the threshold determined by k , while dividing into smaller batches is more efficient when processing time scales more rapidly with batch size.

In practice, our system implements this insight by evaluating different batch sizes at runtime based on samples collected during the active profiling phase. For example, when there are 64 requests in the queue, we assess the latency for batch sizes of 8, 16, 32, and 64 using the above formulas to determine the best configuration. Note that retrieval and generation workers adopt different batch scheduling strategies due to their distinct characteristics. Retrieval tasks typically perform well with larger batches, as they primarily rely on the CPU for vector operations. In contrast, LLM inference offloads more tensors to lower memory hierarchies with larger batches, causing processing time to scale more rapidly with batch size. Our approach accounts for these differences by optimizing each pipeline independently based on its performance characteristics.

5 Implementation

We have implemented a prototype of RAGDOLL in approximately 5000 lines of code. Our knowledge base is built upon the Document class in LangChain [34], a widely used framework for integrating LLMs into applications. For storage, we use Milvus [59], a popular vector database recognized by the GitHub community [43], to manage document chunks and associated embeddings. Additionally, our LLM prefetching module is built on FlexGen [53], which is specifically designed for offloading-based inference on a single GPU.

Lazy dynamic transfer. Given our adaptive computation workspace and dynamic memory hierarchy, it is essential to transfer tensors during LLM inference when policy changes occur. We adopt a lazy transfer approach: rather than transferring model weights between CPU and GPU memory synchronously, we perform these transfers on a background CUDA stream and only synchronize when a weight tensor is imminently required for computation. Moreover, once a weight tensor has been offloaded to disk, we retain its unique file name and reuse it across generations, ensuring that each weight tensor incurs an offload cost only once. Since our weight placement policy adjusts incrementally, only a small subset of tensors require transfer during policy updates—this mechanism applies to both profiling and runtime phases. For KV cache, we clear all cached data from the memory hierarchy and reinitialize it at the start of each generation,

following the same pattern used for hidden states and attention masks. Similarly, for vector database partitions, we adopt a lazy strategy by releasing and loading partitions during the retrieval phase, thereby keeping the overall cost of policy transfer minimal.

Fault tolerance. Despite provisioning for additional memory on both the GPU and CPU, resource constraints may still lead to out-of-memory (OOM) errors. To mitigate this, we have incorporated fault-tolerant mechanisms in both our offline profiling and online runtime phases. We checkpoint intermediate retrieval chunks for each partition; should a retrieval failure occur, the system reverts to the most recent successful checkpoint. For LLM inference, a checkpoint is maintained at the beginning of each generation. In the event of a GPU or CPU OOM during generation, we first attempt to reinitialize the KV cache by shifting more tensors to a lower memory tier; if that is insufficient, we offload a portion of the weight tensors to lower memory. This design minimizes the need for a full restart and reduces the overhead associated with additional I/O operations.

6 Evaluation

In this section, we evaluate the performance of RAGDOLL compared to baseline systems under various experimental configurations, including different hardware setups, model sizes, and workloads.

6.1 Experimental Setup

Hardware. The experiments are conducted on two platforms representing relatively high- and low-end scenarios:

- **PF-High:** Intel Xeon 4314 processor (63 cores at 2.4 GHz), 256 GB of host memory, an NVIDIA A30 GPU (24 GB), an 8 TB SSD, and PCIe 4.0 interface (64 GB/s bandwidth).
- **PF-Low:** Intel Xeon 6238R processor (111 cores at 2.2 GHz), 176 GB of host memory, an NVIDIA A5000 GPU (12 GB), a 2 TB SSD, and PCIe 3.0 interface (32 GB/s bandwidth).

Dataset and Models. For the knowledge base, we construct a vector database by chunking and embedding documents from the TriviaQA dataset [31], which was selected for its high document volume. TriviaQA is a reading comprehension dataset containing over 650,000 question and answer tuples. The vector database is partitioned into 32 segments, with each partition being approximately 8 GB in size, resulting in a total database size of approximately 256 GB. For the LLMs, we adopt two representative sizes: Llama 3.1 with 8 billion and 70 billion parameters [58]. We chose this range (from 8B to 70B) because these model sizes are not only popular among LLMs as evidenced by leaderboard [33, 38], but also because they can be reasonably executed on a single consumer-grade GPU while allowing most baseline methods to function normally for fair comparison.

Workloads. Each request in our workload is synthesized from a question in the TriviaQA dataset. Under the default

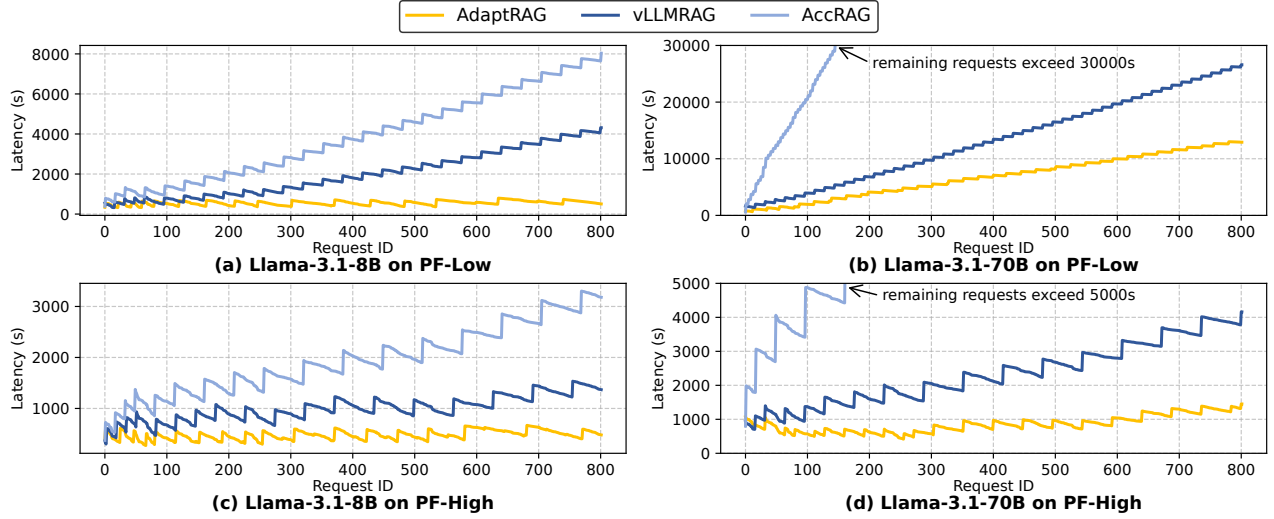


Figure 7. Request latency under a dynamic workload. On the x-axis, the arrival rate increases from left to right: approximately 0–80 indicates 4 requests per minute, 80–240 corresponds to 8 requests per minute, 240–480 to 12 requests per minute, and 480–800 to 16 requests per minute. The linear segments between two distinct turning points represent a generation batch, since nearly uniform arrival times lead to similar completion times and thus a linear latency profile.

settings, the top 5 most relevant document chunks are retrieved from the vector database and concatenated with the original question to form the final input for the LLM. Since the dataset does not include timestamps, we simulate request arrival times using a Poisson distribution with varying request rates, following [33, 66]. The entire workload is divided into 20-minute intervals, with each interval assigned a specific request rate. By default, we use four intervals with request rates of 4, 8, 12, and 16 requests per minute, covering both sustainable and overload scenarios.

Baselines. As discussed in Section 3.2, most existing RAG optimization systems [30, 64] are incompatible with our experimental setup due to two critical limitations: (1) they require loading the entire model weights into VRAM, and (2) they are not designed to handle the large-scale on-disk knowledge bases central to our evaluation. Therefore, we mainly compare our approach against two modern LLM systems with integrated RAG capabilities:

- **RAG with Accelerate (AccRAG).** Hugging Face Accelerate [23] is a well-established system for serving LLMs. It only offloads partial model weights and maintains KV caches on GPU. To ensure sufficient memory for the compute workspace and cache storage, we hereby limit the allocation for weight initialization.
- **RAG with vLLM (vLLMRAG).** vLLM [33] is a state-of-the-art LLM serving system to minimize memory fragmentation and is commonly used in RAG systems [30, 59]. We utilize vLLM v0.7.2, which effectively offloads model weights to RAM and manages computational workspace with the feasible batch size.

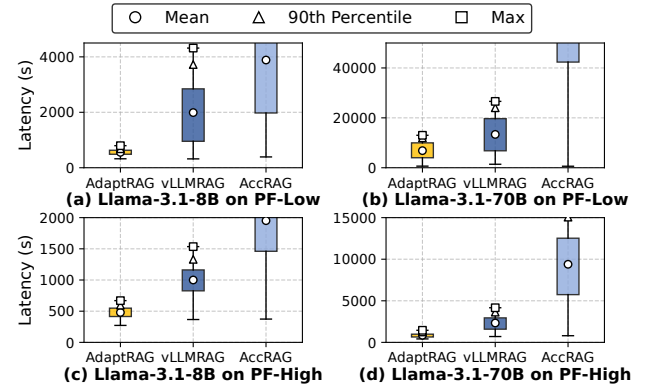


Figure 8. Boxplot of all latency values.

We integrate the baseline LLM servings with RAG in a serial mode, processing batches in their arrival order with an adaptive size $4\lambda(t)$ for each interval. On PF-Low, both baselines encounter memory limitations when serving the 70B model, as the combined GPU and CPU memory is insufficient. To ensure fair comparison, we configure an extended swap space of approximately 180 GB on SSD for PF-Low when integrating the 70B models. To prevent severe performance degradation from excessive I/O operations, we carefully profile and determine the maximum batch sizes that offer optimal throughput and latency performance under memory constrained conditions for the baselines.

Key Metrics. We target online RAG systems by measuring each request’s end-to-end latency, defined as the duration from the request’s arrival to its completion. Additionally, we report average latency and various percentile latencies

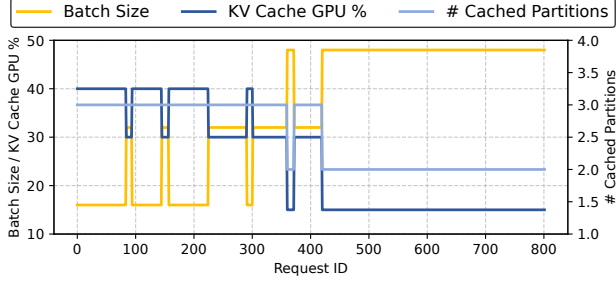


Figure 9. Changes of scheduling of Figure 7(d).

across all requests in selected experiments. Since our proposed systems maintain functional equivalence—preserving the accuracy and quality of outputs without introducing approximations or truncations—our primary focus is on optimizing latency while ensuring consistent result quality.

6.2 End-to-end Performance

As shown in Figure 7, we report the end-to-end latency of each request during the dynamic workload. RAGDOLL consistently demonstrates lower average end-to-end latency than the two baselines, achieving a $1.9\times$ (1236 seconds vs. 2331 seconds in Figure 7(d)) to $3.6\times$ speedup (555s vs. 1990s in Figure 7(a)) compared to vLLMRAG. Moreover, the speedup exceeds $11.7\times$ when compared to Accelerate on large models with limited resources (see Figure 7(b)).

For the 8B model, RAGDOLL maintains an upper bound for each request without experiencing latency explosion under higher workloads. For example, in Figure 8(a), every request’s latency remains below 1000s, and the trend does not increase as arrival rates rise. In contrast, both baseline methods show increasing latency as arrival rates increase. Although vLLMRAG in Figure 7(c) shows a slower upward trend, its latency still increases from 700–800s (before the 100th request when the arrival rate is 4 requests per minute) to approximately 1500s (as shown in Figure 7(c)) after the 700th request when the arrival rate reaches 16 requests per minute. The performance advantage of RAGDOLL can be attributed to its more dynamic batch scheduling policy. For instance, in Figure 7(a), RAGDOLL utilizes notably larger batch sizes, as evidenced by the longer transition segment lengths. Although we also implemented dynamic batch sizing for vLLMRAG, it cannot support the designated larger batch sizes due to limited GPU memory when arrival rates are high. Furthermore, it lacks the ability to pipeline retrieval and generation operations, leading to exponential growth in latency for backlogged requests under limited resources. As for AccRAG, since it does not incorporate prefetching techniques as noted in [23], its generation efficiency is lower, resulting in easier system overload.

For the 70B model, although all methods encounter high arrival rates on both machines, RAGDOLL’s latency escalates more gradually. On PF-Low, where GPU and CPU resources are constrained for 70B model, all methods must use smaller

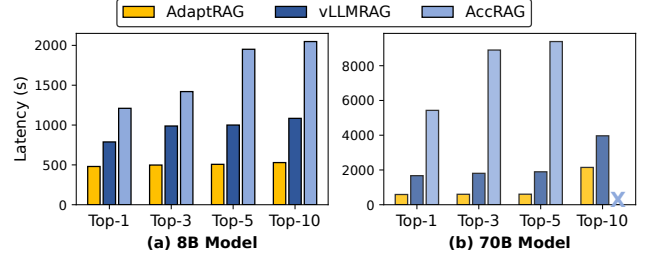


Figure 10. Average latency with different top-k values in retrieval. The “X” mark indicates extreme high latency due to the method’s poor performance.

batch sizes, while RAGDOLL relies on its pipeline architecture and balanced configuration to mitigate the increase. Figure 8(b) and (d) show that RAGDOLL reduces maximum latency by 50% vs. vLLMRAG and by 80% vs. AccRAG, demonstrating superior resilience under heavy workloads.

To quantify the adaptive behavior of RAGDOLL, we analyze the runtime policy adjustments for the 70B model on PF-High as depicted in Figure 9. The data reveals systematic policy transitions as workload intensity increases: the generation batch size progressively scales from 16 to 48, while KV cache CPU allocation decreases from 40% to 15%, and vector database partitions cached in CP reduce from 3 to 2. These coordinated policy shifts respond to increasing backlogs under heavier workloads. Following our adaptive scheduling principles, the system prioritizes throughput by expanding batch sizes, which necessitates additional GPU memory allocation. This memory pressure triggers more KV cache offloading to CPU, which in turn requires releasing vector database partitions to disk to maintain memory balance.

The policy selections exhibit some fluctuations during certain phases. Before the 200th request, the generation batch size occasionally varies between 16 and 32 sometimes. These fluctuations arise because RAGDOLL’s decision making is based on the actual system state rather than simply arrival rate. For instance, when incoming requests are still in the retrieval phase despite high arrival rates, the context queue may remain relatively small, temporarily allowing smaller generation batch sizes. However, as system load stabilizes beyond the 400th request, batch size selection becomes more consistent due to the persistent backlog in the context queue. This behavior demonstrates RAGDOLL’s adaptive capabilities, enabling efficient resource management without requiring prior knowledge of workload characteristics.

6.3 Latency Breakdown

We break down the average latency as shown in Table 1 to examine each component’s contribution. Notably, the waiting time of RAGDOLL is significantly less than the baselines, while its retrieval and generation times are not necessarily better than vLLMRAG. For instance, with the 8B model on PF-Low, RAGDOLL’s waiting time is only 10% of vLLMRAG (170s vs. 1640s), though its retrieval and generation times are

Table 1. Average latency breakdown in seconds.

Machine	Model	Method	Waiting	Retrieval	Generation
PF-Low	8B	RAGDOLL	170	320	66
		vLLMRAG	1640	293	57
		AccRAG	3421	288	176
	70B	RAGDOLL	5895	494	466
		vLLMRAG	12761	376	222
		AccRAG	79715	357	489
PF-High	8B	RAGDOLL	162	282	36
		vLLMRAG	677	307	16
		AccRAG	1494	307	151
	70B	RAGDOLL	606	388	242
		vLLMRAG	1808	303	219
		AccRAG	7936	302	1152

slightly higher. This advantage stems from our larger design space and more efficient workload scheduling. Specifically, when workload intensity shifts from low to heavy, we can dynamically arrange larger batch sizes and release more vector database partitions. Although a single request might experience longer retrieval and generation latency, the total latency is reduced through the optimizations analyzed in Section 4.4. Additionally, while pipelined retrieval and generation processes may interfere with each other, our pipelined design overlaps substantial latency components, reducing the average latency across all requests. In contrast, vLLMRAG, despite its efficiency in inference through effective swapping optimization, lacks adaptive scheduling capabilities and thus cannot efficiently manage overall requests, resulting in high waiting latency. Compared to AccRAG, RAGDOLL reduces waiting and generation times by up to 20× and 5×, mainly because AccRAG lacks offloading techniques like prefetching, which leads to significantly longer generation times and accounts for its high overall latency.

Table 2. Ablation study of proposed techniques. We report the average latency (seconds) on PF-High. The gray number is the static generation batch size policy in the whole process.

Method	8B	70B
RAGDOLL	480	1236
Without pipelined design	663	1954
Without dynamic batch size	657 (64)	1841 (32)
Using FlexGen for LLM inference module	531	1283
Using vLLM for LLM inference module	561	1432

6.4 Ablation Study

To quantify the contribution of each technique in our system, we conduct a comprehensive ablation study. Table 2 presents the latency impact when disabling individual components of RAGDOLL. When disabling the pipelined design, we observe the most significant performance degradation: a 38% increase in latency for the 8B model (663s vs. 480s) and a 58% increase for the 70B model (1954s vs. 1236s). This substantial impact occurs because, despite retaining other optimizations such as dynamic batch sizing and memory allocation, system flexibility becomes severely constrained. Specifically, the retrieval and generation workers must use identical batch

sizes, making it challenging to optimize for both stages simultaneously. Furthermore, both components experience idle waiting periods, reducing overall throughput.

Disabling the dynamic batch size policy also leads to considerable performance deterioration: 657s vs. 480s for the 8B model and 1841s vs. 1236s for the 70B model. This decline stems from the need to pre-allocate memory for the maximum designated batch size, which sacrifices efficiency during periods of low request arrival rates and introduces additional waiting time.

We further evaluate our LLM prefetching module by replacing it with alternatives. First, we integrate a modified version of FlexGen made compatible with Llama models. Despite maintaining all other optimizations, this configuration still results in 10% and 4% higher latency than RAGDOLL for the 8B and 70B models, respectively. The superior performance of RAGDOLL’s prefetching module is attributed to its pipelined communication and computation design, which maximizes GPU and CPU utilization. The smaller improvement observed for the 70B model occurs because a larger proportion of LLM tensors reside in lower memory hierarchies, making I/O the dominant bottleneck and limiting the additional overlap our prefetching module can achieve.

We also assess the integration of vLLM into our system. While this configuration exhibits significantly better latency than a serial implementation of vLLM in a RAG system (vLLMRAG), achieving 561s vs. 1000s for the 8B model, and 1432s vs. 2331s for the 70B model. However, it still falls short of the performance achieved by RAGDOLL. Although vLLM can accommodate different batch sizes, it does so within the constraint of fixed weight tensor distribution between GPU and CPU memory. Specifically, while vLLM’s interface provides flexibility for a wide range of batch size adjustments, its internal implementation actually determines the effective batch size based on memory constraints rather than directly honoring our specified parameters. Consequently, when batch sizes grow larger, the latency increases almost linearly with batch size, which becomes suboptimal, as discussed in Section 4.4. This issue, while present to some degree in both FlexGen and RAGDOLL, is more pronounced in vLLM. Nevertheless, despite its constrained dynamic capabilities, vLLM can still benefit from our pipelined design and reasonable tuning, enabling it to achieve substantially better performance than vLLMRAG.

In conclusion, we opt for an offloading-based LLM inference system like FlexGen because it offers greater flexibility in dynamically adjusting model weights and KV cache placement across the memory hierarchy. This allows for better trade-offs between throughput and latency under different workloads. However, RAGDOLL’s implementation can effectively accommodate other types of LLM inference systems such as vLLM, demonstrating the excellent compatibility and extensibility for diverse backend integration.

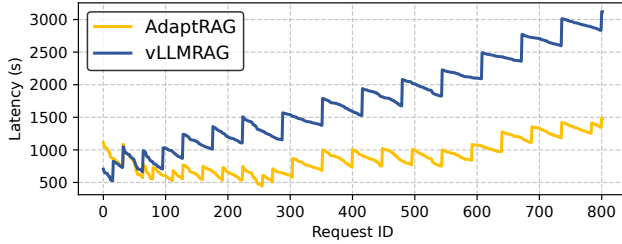


Figure 11. Performance of 70B model under DiskANN.

6.5 Case Study

Number of Retrieval Chunks. We evaluate the impact of different top-k chunks in retrieval. Changes in retrieval chunk count do not significantly affect retrieval latency because loading costs substantially outweigh search costs in the retrieval process, making the impact of additional chunks minimal. However, chunk count substantially impacts generation latency by altering input sequence length. For the 8B model, where generation is not the dominant component, RAGDOLL’s latency increase remains modest, rising from 480s (for top-1 with input token length under 128) to 529s (for top-10 with input token length under 1024). Similarly, vLLMRAG maintains relative stability (788s to 1080s) because vLLM can maintain the designated batch size for the 8B model on PF-High even as input length approaches 1024 tokens. However, both RAGDOLL and vLLMRAG experience significant latency growth with top-10 retrieval for larger models, as generation latency becomes dominant. In these scenarios, neither method can utilize larger batch sizes. Nevertheless, RAGDOLL still maintains a 1.8x speedup over vLLMRAG (2147s vs. 3966s), demonstrating that even when batch size adjustments are constrained, RAGDOLL’s pipelined design achieves substantially better performance.

On-Disk Index. We also evaluated our method’s performance using an on-disk index by creating the DiskANN [24] index of the vector database with Milvus’s default configuration. In this setup, searching each partition requires approximately 3.6 GB of CPU memory to store product quantization codes and cache. Consequently, even on PF-High, the system cannot simultaneously accommodate both the index and LLM in memory, necessitating a trade-off in memory allocation. As illustrated in Figure 11, the latency of RAGDOLL under DiskANN is better than that in the same hardware setup and LLM configuration shown in Figure 7(d) (average 890s vs. 1236s). This improvement is due to RAGDOLL’s active profiling, which effectively balances the retrieval and generation processes, allowing for reasonable memory allocation between the index and LLM tensors, thereby leveraging DiskANN to enhance retrieval. In contrast, vLLMRAG does not exhibit a noticeable improvement and performs slightly worse (2427s vs. 2331s) under the same setup. This is because, although the DiskANN index saves more memory

than the original data, reloading the index for each partition is more time-consuming than loading the original data due to additional disk I/O operations and index initialization overhead. To fully utilize the on-disk index, reserving more memory for the index is necessary, which may impact the efficiency of LLM inference. This underscores the complexity of optimizing scheduling and allocation policies in RAG systems under resource constraints. In summary, while the patterns differ between using and not using an on-disk index for retrieval, we believe this can be viewed as a shift in trade-offs between retrieval and generation, allowing RAGDOLL to optimize accordingly. Moreover, employing an on-disk index can enhance the scalability of the system by enabling efficient processing of much larger knowledge bases. This demonstrates the practical value of our approach for deployment in realistic production environments.

7 Related Work

Efficient LLM Serving. To enable efficient LLM inference under strict performance and resource constraints, numerous systems [4, 15, 16, 28, 51, 57, 62, 68] improve inference efficiency by offloading KV caches or model weights to limit GPU memory usage. A line of research [53, 55, 67] represented by FlexGen prioritize inference throughput. In comparison, systems like vLLM [33] particularly enhance online latency through batch scheduling techniques. RAGDOLL is compatible with many of these optimization strategies. In this paper, we adopt FlexGen as our representative implementation due to its flexible tensor placement capabilities, which align well with our focus on resource-limited settings. Another approach to reduce LLM size is model compression [12, 18, 63, 65], since it does not inherently address the memory issue, it is commonly regarded as an auxiliary solution alongside offloading methods in practical deployments.

LLM Computation for RAG. RAG [19, 35, 36, 42, 52] enhances LLM generation by incorporating external knowledge. Recent studies [25–27, 30, 39, 64, 69] dedicate to more efficient LLM serving tailored for RAG, primarily focusing on the management of KV cache inside generation computation. RAGDOLL is complementary to these techniques with the capability in jointly optimizing retrieval and generation under both computation and storage constraints.

On-disk Vector Retrieval. Production-grade vector databases [11, 13, 43, 49] offer fundamental retrieval capabilities for RAG systems. Indexing algorithms [21, 24, 54] further power data retrieval by constructing on-disk indices to boost query lookup and reduce memory overhead. RAGDOLL complements these approaches by systematically optimizing their efficiency under resource-constrained RAG deployments.

8 Conclusion

We present RAGDOLL, a self-adaptive LLM-RAG integration on a consumer-grade GPU. By implementing joint memory management, enhanced multi-pipeline processing, and adaptive batch scheduling, our system achieves up to 3.6× speedup compared to state-of-the-art baselines. These results demonstrate that carefully designed resource allocation strategies can make large-scale RAG practical on resource-constrained devices.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Amey Agrawal, Anmol Agarwal, Nitin Kedia, Jayashree Mohan, Souvik Kundu, Nipun Kwatra, Ramachandran Ramjee, and Alexey Tumanov. 2024. Etalon: Holistic Performance Evaluation Framework for LLM Inference Systems. *arXiv preprint arXiv:2407.07000* (2024).
- [3] Amey Agrawal, Anmol Agarwal, Nitin Kedia, Jayashree Mohan, Souvik Kundu, Nipun Kwatra, Ramachandran Ramjee, and Alexey Tumanov. 2024. Metron: Holistic performance evaluation framework for llm inference systems. *arXiv e-prints* (2024), arXiv–2407.
- [4] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. DeepSpeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [5] Akari Asai, Sewon Min, Zexuan Zhong, and Danqi Chen. 2023. Retrieval-based language models and applications. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 6: Tutorial Abstracts)*. 41–46.
- [6] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374.
- [7] Jordi Bayarri-Planas, Ashwin Kumar Gururajan, and Dario Garcia-Gasulla. 2025. Pareto-Optimized Open-Source LLMs for Healthcare via Context Retrieval. *arXiv:2409.15127 [cs.AI]* <https://arxiv.org/abs/2409.15127>
- [8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [9] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024. Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 17754–17762.
- [10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [11] Chroma. 2025. <https://www.trychroma.com/>.
- [12] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems* 35 (2022), 30318–30332.
- [13] Etienne Dlocker, Bob van Luijt, Byron Voorbach, Mohd Shukri Hasan, Abdel Rodriguez, Dirk Alexander Kulawiak, Marcin Antas, and Parker Duckworth. [n. d.]. *Weaviate*. <https://github.com/weaviate/weaviate> If you use this software, please cite it as below..
- [14] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library. *arXiv preprint arXiv:2401.08281* (2024).
- [15] Hongchao Du, Shangyu Wu, Arina Kharlamova, Nan Guan, and Chun Jason Xue. 2025. FlexInfer: Breaking Memory Constraint via Flexible and Efficient Offloading for On-Device LLM Inference. In *Proceedings of the 5th Workshop on Machine Learning and Systems*. 56–65.
- [16] Artyom Eliseev and Denis Mazur. 2023. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238* (2023).
- [17] Tianyu Fan, Jingyuan Wang, Xubin Ren, and Chao Huang. 2025. Mini-RAG: Towards Extremely Simple Retrieval-Augmented Generation. *arXiv preprint arXiv:2501.06713* (2025).
- [18] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [19] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997 2* (2023).
- [20] Github. 2022. <https://github.com/features/copilot>.
- [21] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Nee-lam Mahapatro, Premkumar Srinivasan, et al. 2023. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*. 3406–3416.
- [22] Google. 2023. <https://bard.google.com/>.
- [23] HuggingFace Accelerate. 2022. <https://huggingface.co/docs/accelerate/index>.
- [24] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information processing systems* 32 (2019).
- [25] Wenqi Jiang, Suvinay Subramanian, Cat Graves, Gustavo Alonso, Amir Yazdanbakhsh, and Vidushi Dadu. 2025. RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving. *arXiv preprint arXiv:2503.14649* (2025).
- [26] Wenqi Jiang, Marco Zeller, Roger Waleffe, Torsten Hoefer, and Gustavo Alonso. 2023. Chameleon: a heterogeneous and disaggregated accelerator system for retrieval-augmented language models. *arXiv preprint arXiv:2310.09949* (2023).
- [27] Wenqi Jiang, Shuai Zhang, Boran Han, Jie Wang, Bernie Wang, and Tim Kraska. 2024. Piperag: Fast retrieval-augmented generation via algorithm-system co-design. *arXiv preprint arXiv:2403.05676* (2024).
- [28] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2024. Neo: Saving gpu memory crisis with cpu offloading for online llm inference. *arXiv preprint arXiv:2411.01142* (2024).
- [29] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *Advances in Neural Information Processing Systems* 37 (2024), 59532–59569.
- [30] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457* (2024).
- [31] Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551* (2017).
- [32] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 453–466.

- [33] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [34] LangChain. 2022. <https://github.com/langchain-ai/langchain>.
- [35] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [36] Huayang Li, Yixuan Su, Deng Cai, Yan Wang, and Lema Liu. 2022. A survey on retrieval-augmented text generation. *arXiv preprint arXiv:2202.01110* (2022).
- [37] LlamaIndex. 2022. <https://www.llamaindex.ai/>.
- [38] LMSYS ORG. 2023. Chatbot Arena Leaderboard Week 8: Introducing MT-Bench and Vicuna-33B. <https://lmsys.org/blog/2023-06-22-leaderboard/>.
- [39] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. 2024. TurboRAG: Accelerating Retrieval-Augmented Generation with Precomputed KV Caches for Chunked Text. *arXiv preprint arXiv:2410.07590* (2024).
- [40] Ziyang Luo, Can Xu, Pu Zhao, Xiubo Geng, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. Augmented large language models with parametric knowledge guiding. *arXiv preprint arXiv:2305.04757* (2023).
- [41] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [42] Yuning Mao, Pengcheng He, Xiaodong Liu, Yelong Shen, Jianfeng Gao, Jiawei Han, and Weizhu Chen. 2020. Generation-augmented retrieval for open-domain question answering. *arXiv preprint arXiv:2009.08553* (2020).
- [43] Milvus. 2022. <https://github.com/milvus-io/milvus>.
- [44] John X Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M Rush. 2023. Text embeddings reveal (almost) as much as text. *arXiv preprint arXiv:2310.06816* (2023).
- [45] OpenAI. 2022. <https://openai.com/blog/chatgpt>.
- [46] Bowen Pang, Kai Li, and Feifan Wang. 2025. Optimizing LLM Inference Throughput via Memory-aware and SLA-constrained Dynamic Batching. *arXiv preprint arXiv:2503.05248* (2025).
- [47] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [48] Archit Patke, Dharmath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Shengkun Cui, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishankar Iyer. 2024. One queue is all you need: Resolving head-of-line blocking in large language model serving. *arXiv preprint arXiv:2407.00047* (2024).
- [49] Pinecone Systems, Inc. 2025. Pinecone. <https://www.pinecone.io/>. Accessed: 2025-04-03.
- [50] Sonal Prabhune and Donald J Berndt. 2024. Deploying Large Language Models With Retrieval Augmented Generation. *arXiv preprint arXiv:2411.11895* (2024).
- [51] Alexander Raistrick, Lingjie Mei, Karhan Kayan, David Yan, Yiming Zuo, Beining Han, Hongyu Wen, Meenal Parakh, Stamatis Alexandropoulos, Lahav Lipson, et al. 2024. Infinigen indoors: Photorealistic indoor scenes using procedural generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 21783–21794.
- [52] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics* 11 (2023), 1316–1331.
- [53] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
- [54] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. Freshdiskann: A fast and accurate graph-based ann index for streaming similarity search. *arXiv preprint arXiv:2105.09613* (2021).
- [55] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 590–606.
- [56] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2024. Dynamollm: Designing llm inference clusters for performance and energy efficiency. *arXiv preprint arXiv:2408.00741* (2024).
- [57] Yifan Tan, Cheng Tan, Zeyu Mi, and Haibo Chen. 2024. PipeLLM: Fast and Confidential Large Language Model Services with Speculative Pipelined Encryption. *arXiv preprint arXiv:2411.03357* (2024).
- [58] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [59] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.
- [60] Yuxin Wang, Yuhang Chen, Zeyu Li, Xueze Kang, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, et al. 2024. BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems. *arXiv preprint arXiv:2401.17644* (2024).
- [61] Zilong Wang, Zifeng Wang, Long Le, Huaixiu Steven Zheng, Swaroop Mishra, Vincent Perot, Yuwei Zhang, Anush Mattapalli, Ankur Taly, Jingbo Shang, et al. 2024. Speculative rag: Enhancing retrieval augmented generation through drafting. *arXiv preprint arXiv:2407.08223* (2024).
- [62] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 640–654.
- [63] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.
- [64] Jiayi Yao, Hanchen Li, Yuhang Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2024. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. *arXiv preprint arXiv:2405.16444* (2024).
- [65] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. 2022. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *Advances in Neural Information Processing Systems* 35 (2022), 27168–27183.
- [66] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.

- [67] Xuanlei Zhao, Bin Jia, Haotian Zhou, Ziming Liu, Shenggan Cheng, and Yang You. 2024. Hetegen: Heterogeneous parallel inference for large language models on resource-constrained devices. *arXiv preprint arXiv:2403.01164* (2024).
- [68] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems* 37 (2024), 62557–62583.
- [69] Yun Zhu, Jia-Chen Gu, Caitlin Sikora, Ho Ko, Yinxiao Liu, Chu-Cheng Lin, Lei Shu, Liangchen Luo, Lei Meng, Bang Liu, et al. 2024. Accelerating inference of retrieval-augmented generation via sparse context selection. *arXiv preprint arXiv:2405.16178* (2024).